

S. NO	DATE	Title	Sign.
1.	21/07/25	BASIC NETWORKING COMMANDS	21/7/25 (10)
2.	24/07/25	NETWORK CABLES	24/7/25 (10)
3.	3/08/25	PACKET TRACER TOOL INSTALLATION AND USER INTERFACE	3/8/25 (10)
4.	4/08/25	WIRESHARK.	4/8/25 (10)
5.	1/9/25	HAMMING CODE	1/9/25 (10)
6.	1/9/25	SLIDING WINDOW.	1/9/25 (10)
7.	8/9/25	Setup LAN using Switch and Ethernet	8/9/25 (10)
8.	15/9/25	NMAP on the TryHackme platform	15/9/25 (10)
9.	29/9/25	Subnetting in cisco packet Tracer	29/9/25 (10)
10.	3/10/25	Internetworking with Routers in cisco packet Tracer Simulator.	3/10/25 (10)
11.	5/10/25	Simulate static Routing configure using cisco packet Tracer.	5/10/25 (10)
12.	7/10/25	a) Simulate static Routing protocol Config using cisco packet Tracer b) Simulate RIP using cisco	7/10/25 (10)
13.	8/10/25	Implement your own ping prog. socket	8/10/25 (10)
14.		RAW Sockets to implement packet sniffing.	
15.		Analyse various types of servers using nmap tool	

Exp. 1

21/07/25

CS19541 - COMPUTER NETWORKS

AIM

Exp 1 :- Study of Various Network Commands used in LINUX and windows.

BASIC NETWORKING COMMANDS (windows)

- ① `arp -a` : shows IP address of your computer along with the IP and MAC address of your route

OUTPUT:-

Interface : 172.16.75.34 --- OX 7

Internet Address

Physical Address

Type

172.16.72.1

7c-5a-1c-cf-be-4a : dynamic

172.16.72.42

CC-47-40-8d-0d-af dynamic

- ② `hostname` : Displays the name of your computer

output :

DESKTOP - 8938VQ1

- ③ `ipconfig /all` : Displays Important Configuration Information about your TCP/IP Connection.

output :

windows IP Configuration

Hostname

DESKTOP - 8938VQ1

Primary Dns Suffix

Node Type

Hybrid

- ⑨ Router : Views or modifies the IP routing table
used to add / Delete static routes.

OUTPUT :-

Interface List

16... 20 88 10 86 7a be... Intel(R) Ethernet Connect
2... 4e 82 a9 78 9b e7... microsoft wifi Direct
Virtual Adapter

SOME IMPORTANT LINUX NETWORKING COMMANDS.

- ① Ip : Basic Command Every administration will
need in daily work.

→ Ip address show.

OUTPUT :-

1. lo : < LOOPBACK , UP , LOWER_UP >

:

2. enpos31f6 :

3. wlan0

Pnet 172.16.75.125 / 21

→ Sudo Ip address add 192.168.1.254 / 24 dev enpos31f6

OUTPUT :

Connection established.

You are now connected to enpos31f6

→ sudo ip address del 192.168.1.254 / 24 dev enpos31f6

- sudo ip link set enpos31f6 up
- sudo ip link set enpos31f6 down
- sudo ip link set enpos31f6 promote
- sudo ip route add default vrf 192.168.1.254 dev enpos31f6

② ifconfig.

The ifconfig cmd is a staple in many sysadmin's tool belt for configuring and troubleshooting networks.

```

OUTPUT
enpos31f6 : flags = 4355 <UP, BROADCAST, ...>
           mtu 1500
           ipnet 192.168.1.100

```

```

wlp2s0 : flags = 73 <UP, LOOPBACK, RUNNING>

```

```

wlp2s0 : flags = 4163 <...>

```

```

ipnet 172.16.75.125

```

③ mtr

MTR (Matt's traceroute) is a program with a command line interface that serves as a network diagnostic and troubleshooting tool. This command combines the functionality of the ping and traceroute commands.

Syntax of the Command

mtr < options > hostname / IP

Common use cases

a) mtr google.com.

OUTPUT

Host	packets	pings			
	loss %	snt	last	Avg	Best
1. gateway	0.0 %	85	2.4	7.5	1.7
2. static_41.229.249...	0.0 %	85	66.9	14.0	4.4
:	:	:	:	:	:

b) mtr -n google.com

Host

packets

pings

	loss %	snt	last	Avg	Best...
1. 172.16.72.1	11.9 %	163	8.3	28.2	2.4
2. 49.249.229.41	11.6 %	66	6.7	41.6	4.9
:	:	:	:	:	:

c) mtr -b google.com.

d) mtr -c 10 google.com.

OUTPUT

Host

packets

Pings

	loss %	snt	last	Avg	Best	Worst
1. gateway	0.0 %	2	6.2	8.9	6.1	239.4
2. static_41.229.249...	0.0 %	2	5.5	11.0	22.1	72.2
:	:	:	:	:	:	:

1) tcpdump

The tcpdump cmd is designed for capturing and displaying packets

→ Install tcpdump with the Command :

1. sudo apt install -y tcpdump

OUTPUT :

:

package tcpdump-14.4.99.4-2-rc39.x86 is already installed.

dependencies resolved.

Nothing to do

Complete!

2. tcpdump - D

OUTPUT :

1. wlan0 [UP, Running, wireless, Associated]

2. Any

3. lo [UP, Running, loopback]

4. enp0s31f6 [up, Disconnected]

:

3. sudo tcpdump - i wlan0

listening on wlan0...

5 packets captured.

481 packets received by

35% packets dropped by kernel.

4. `sudo tcpdump -i wlan0 -c 10`
OUTPUT:

Listening on wlan0, link-type EN10MB

:

10 packets captured.

39 packets received by filter

0 packets dropped by kernel.

5) To find traffic coming from and going to 8.8.8.8

use the command:

`tcpdump -i wlan0 -c 10 host 8.8.8.8`

OUTPUT:

01:43:22.402561 IP fedora > dns.google

:

10 packets captured

10 packets received by filter

0 packets dropped by kernel

6) For traffic coming from 8.8.8.8,

`sudo tcpdump -i wlan0 src host 8.8.8.8`

OUTPUT:

Listening on wlan0, link-type EN10MB

23:21:22.453708 IP dns.google > fedora

23:21:23.479039 IP dns.google > fedora

:

5) ping :- It is used to troubleshoot connectivity, reachability & name resolution. ping google.com

Output: ping google.com (142.253.221.288) 56 (34) bytes of data.

from fedora (192.168.1.294) icmp-seq = 1

Destination Host unreachable

from fedora (192.168.1.294) icmp-seq = 2.

Destination

Host unreachable.

Student Observation :

1) Which command is used to find the reachability of a host machine from your device

Ans: ping → used to check the reachability of a host
ping <hostname or IP>
ping google.com.

2) Which command will give the details of hops & time taken by packet to reach its destination

Ans: traceroute → shows the path & hops a packet takes to reach a destination.

3) Which command displays the IP configuration of your machine.

Ans: ifconfig or ip address → Displays IP configuration

4) Which cmd displays the TCP port status.

Ans:- `netstat -t` or `ss -t` → Shows TCP port status

5) Write the Command to modify the IP Configuration in a Linux Machine?

Ans: To add : `ip address add 192.168.1.254/24 dev`

To del : `ip address del 192.168.1.254/24 dev`

Destination

Host unreachable

Student Observation

1) Which command is used to find the reachability of a host machine from your device

Ans: `ping` → used to check the reachability of a host

`ping <hostname or ip>`

`ping google.com`

2) Which command will give the details of a network interface card

Ans: `ifconfig` → shows the configuration of a network interface card

Result:-

Therefore, the study of various Network Command used in Linux and Windows are Executed

Successfully

21/11

24/7/25
Exp-2

NETWORK CABLES

Aim :- Study of Different types of Network Cables.

a) understand Different types of Network Cable.

Different types of cable used in networking are:

1. Unshielded Twisted pair (UTP) Cable
2. Shielded Twisted pair (STP) Cable
3. Coaxial Cable
4. Fibre Optic Cable

Cable Type	Category	Maximum Data Transmission	Advantages / Disadvantages	Application / Use
UTP	category 3	10bps	<u>Adv.</u> • cheaper in cost • Easy to install. <u>Disadv.</u> • Susceptible to EMI and noise.	• 10Base-T Ethernet
	category 5	up to 100 Mbps		• Fast Ethernet • Gigabit Ethernet
	category 5e	1 Gbps		• Fast Ethernet
STP	category 6	10 Gbps	<u>Adv.</u> • Shielded • Faster than UTP <u>Disadv.</u> • Expensive • Greater installation effort	• Gigabit Ethernet • 10G Ethernet
SSTP	category 7	10 Gbps		widely used in data centres
Coaxial cable	RG-6 RG-59 RG-11	10-100 Mbps	<u>Adv.</u> • High bandwidth <u>Disadv.</u> • Limited Distance & cost	Speed of signal is soon Television network High Speed Internet connection
Fibre optics cable	Single mode Multi mode	100 Gbps	<u>Adv.</u> • High Speed, Security • Long distance <u>Disadv.</u> • Req skilled installer	Maximum Distance of fibre optic cable is around 100 meters

b) Make Your

STUDENT OBSERVATION

1. What is the Difference btw cross cable and Straight.

Ans

Straight Cable : Same wiring on both ends (used for different devices)

Cross cable : Different wiring on each end (used for similar devices)

2.

Which type of cable is used to connect two PCs?

Ans

Cross cable

3. Which type of cable is used to connect router/switch to PC?

Ans

Straight cable

4. Find out the category of twisted pair cable used in your LAN to connect the PC to the network socket.

Ans

Category 5E (cat. 5e), or Category 6 (cat. 6)

5. Write down your understanding, challenges faced, and a received while making a twisted pair cross / Straight

Ans

- understanding : Arranged 8 wires in T568A/B Std 2 cr into RJ-45.

- challenges : Pre-alignment, correct order, & proper

- output : Cable worked successfully & passed LAN Test

Result

Thus, the Study of different type of Network Cable done successfully.

PACKET TRACER TOOL INSTALLATION AND USER INTERFACE

Aim: To study the packet tool Installation and user Interface Overview.

- a) Install and set up Cisco packet Tracer.
- b) Analyse the behaviour of Network devices using CISCO PACKET TRACER Simulator.

1) From the Network component box, click and drag-and-drop the below components :

- a) 4 Generic PCs and One HUB
- b) 4 Generic PC's and One Switch

2) Click on Connections :

- a) Click on Copper Straight-Through Cable,
- b) Select one of the PC and connect it to HUB using the cable. The link LED should glow in green, indicating that the link is up. Similarly connect remaining 3PCs to the HUB
- c) Similarly connect 4PCs to the switch using copper straight-through cable.

3) Click on the PCs connected to hub, go to the Desktop tab, click on IP Configuration, and enter an IP address and Subnet mask. Here, the default gateway and DNS server information is not needed as there are only two end devices in the Network. Click on the PDU from the common tool bar

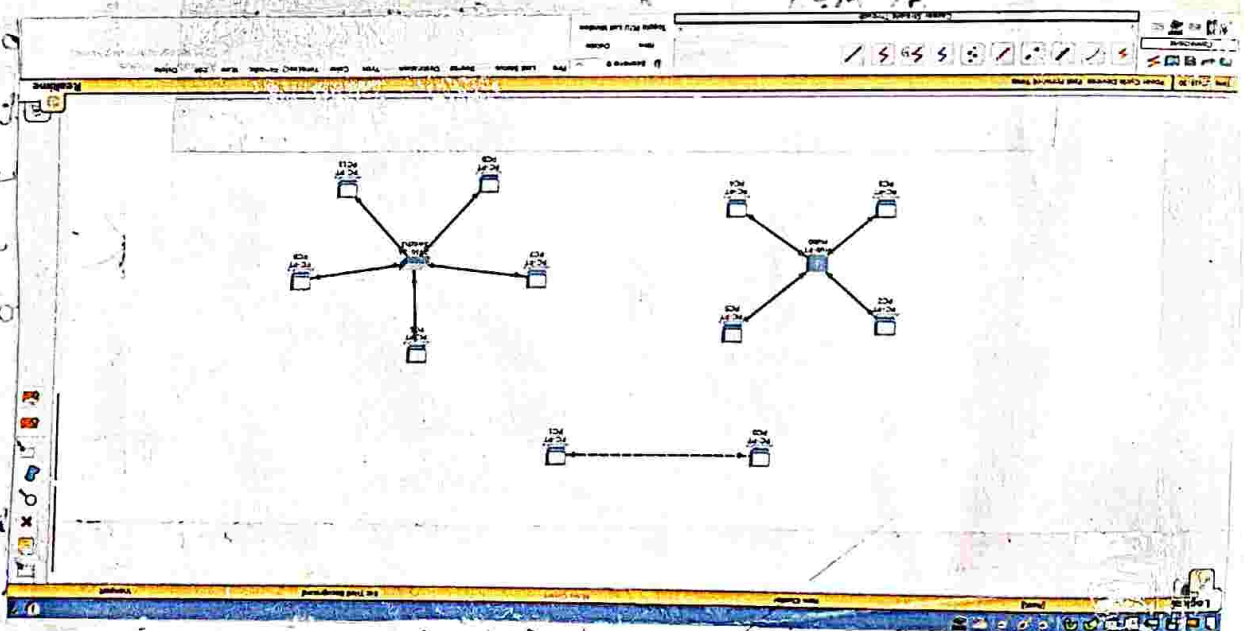
Desktop
IP config
enter IP add. 10.10.10.1
Subnet Mask

a) Drag and drop it on one of PC (Source machine) and then drop it on another PC (Destination machine) connected to the HUB.

4) Observe the flow of PDU from Source PC to destination PC by selecting the Realtime mode of simulation.

5) Repeat step #3 to step #4 for the PCs connected to the Switch.

6) Observe how HUB and Switch are forwarding the PDU and write your observation and conclusion about the behaviors of Switch and HUB.



STUDENT OBSERVATION:

- a) From your observation write down the behavior of Switch and HUB in terms of forwarding the packets received by them.

Ans:

Hub: Broadcasts packets to all ports, causing unnecessary traffic.

Switch: Forwards packets only to the destination port using MAC address, reducing traffic.

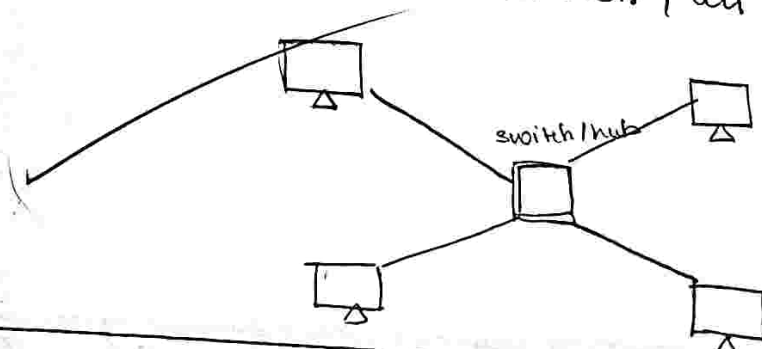
- b) Find out the Network topology implemented in your college and draw and label that topology in your observation book.

Ans:

Network topology

Type: Star topology.

Drawing: One Central Switch, all Computers connected to it.



Result

The packet tracer tool Installation and Use Interface Overview are Successfully Executed.

9/12/23

4/8/25

Exp: 8

WIRESHARK

AIM: Experiments on Packet Capture Tool: Wireshark

1. Create a Filter to display only TCP / UDP packets, inspect the packets, and provide the flow graph

- Select Local Area Connection in Wireshark
- Go to capture → option
- Select stop capture automatically after 100 packets
- Then click Start Capture.
- Search TCP packets in search bar.
- To see flow graph click Statistics → Flow graph.
- Save the packets.

No.	Time	Source	Destination	Protocol	Length	Info
134	11.054431	44.193.52.4	192.168.1.1	TLSv1.2	110	Application Data
135	11.055736	44.193.52.4	192.168.1.8	TLSv1.2	85	Encrypted Alert
137	11.055736	44.193.52.4	192.168.1.8	TCP	54	443 → 52954 [FIN, ACK] Seq=88 Ack=1 Win=213 Len=0
138	11.056003	192.168.1.8	44.193.52.4	TCP	54	52954 → 443 [ACK] Seq=1 Ack=89 Win=513 Len=0
139	11.116725	192.168.1.8	44.193.52.4	TLSv1.2	114	Application Data
140	11.120023	192.168.1.8	44.193.52.4	TCP	54	52954 → 443 [RST, ACK] Seq=1 Ack=89 Win=0 Len=0
141	11.147980	192.168.1.3	255.255.255.255	UDP	78	56700 → 56700 Len=36
142	11.150131	44.193.52.4	192.168.1.8	TCP	54	443 → 52954 [RST] Seq=89 Win=0 Len=0
143	11.836982	192.168.1.8	104.18.41.158	TCP	55	52934 → 443 [ACK] Seq=1 Ack=1 Win=107 Len=1
144	11.866830	104.18.41.158	192.168.1.8	TCP	66	443 → 52934 [ACK] Seq=1 Ack=2 Win=16 Len=0 SLE=1 SRE=2
145	14.696361	fe80::f1d2:2145:e45...	fe80::1	DNS	116	Standard query 0x0f5c A win-extension.fenetrics.grannary.io
146	14.704014	fe80::1	fe80::f1d2:2145:e45...	DNS	212	Standard query response 0x0f5c A win-extension.fenetrics.grannary.io A 44.198.132.62 A 174.61.7...
147	14.705123	192.168.1.8	44.198.132.62	TCP	66	53000 → 443 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 WS=256 SACK_PERM
148	14.825810	44.198.132.62	192.168.1.8	TCP	66	443 → 53000 [SYN, ACK] Seq=0 Ack=1 Win=26 Len=0 MSS=1452 SACK_PERM WS=256
149	14.926027	192.168.1.8	44.198.132.62	TCP	54	53000 → 443 [ACK] Seq=1 Ack=1 Win=132096 Len=0

2. Create a Filter to display only ARP packets and inspect the packets.

- Go to capture → option
- Select stop capture automatically after 100 packets
- Then click start capture.
- Search ARP packets in search bar.
- Save the packets.

No.	Time	Source	Destination	Protocol	Length	Info
142	3.734448	192.168.1.8	3.233.158.26	TLSv1.3	89	Application Data
146	3.735441	192.168.1.8	3.233.158.26	TLSv1.3	109	Application Data
147	3.739907	192.168.1.8	3.233.158.26	TLSv1.3	702	Application Data
148	3.799649	192.168.1.8	3.233.158.26	TCP	1284	52956 → 443 [ACK] Seq=1403 Ack=3529 Win=131584 Len=1230 [TCP PDU reassembled in 149]
149	3.799699	192.168.1.8	3.233.158.26	TLSv1.3	444	Application Data
150	3.938457	Se:5b:5f:02:d9:01	Broadcast	ARP	42	Who has 192.168.1.4? Tell 192.168.1.5
151	6.027181	3.233.158.26	192.168.1.8	TCP	54	443 → 52956 [ACK] Seq=3529 Ack=628 Win=131744 Len=0
152	6.027181	3.233.158.26	192.168.1.8	TLSv1.3	572	Application Data, Application Data
153	6.027181	3.233.158.26	192.168.1.8	TLSv1.3	94	Application Data
154	6.027547	3.233.158.26	192.168.1.8	TLSv1.3	85	Application Data
155	6.029050	3.233.158.26	192.168.1.8	TCP	54	443 → 52956 [ACK] Seq=4087 Ack=3023 Win=34816 Len=0
156	6.029675	3.233.158.26	192.168.1.8	TLSv1.3	363	Application Data
157	5.133711	192.168.1.8	3.233.158.26	TCP	54	52956 → 443 [ACK] Seq=3054 Ack=4396 Win=130560 Len=0
158	4.298750	3.233.158.26	192.168.1.8	TCP	54	443 → 52956 [ACK] Seq=4396 Ack=3054 Win=34816 Len=0
159	6.055543	Se:5b:5f:02:d9:01	Broadcast	ARP	42	Who has 192.168.1.4? Tell 192.168.1.5

Frame 1: 1466 bytes on wire (11720 bits), 1466 bytes captured (11720 bits) on interface \Device\NPF...
 Ethernet II, Src: LiteonTechno_e3:80:45 (e4:aa:ae:e3:80:45), Dst: zte_13:12:ee (dc:71:37:13:12:ee)
 Internet Protocol Version 4, Src: 192.168.1.8, Dst: 142.250.183.174
 Transmission Control Protocol, Src Port: 52956, Dst Port: 443, Seq: 1, Ack: 1, Len: 1412

5. Create a Filter to display only IP/ICMP packets and inspect the packets

- Select Local Area Connection in Wireshark
- Search ICMP/IP packets in search bar.

No.	Time	Source	Destination	Protocol	Length	Info
42	3.430692	172.217.24.142	192.168.1.8	TLSv1.2	348	Application Data
43	3.430960	172.217.24.142	192.168.1.8	TLSv1.2	93	Application Data
44	3.431060	192.168.1.8	172.217.24.142	TCP	54	52248 → 443 [ACK] Seq=3306 Ack=1337 Win=512 Len=0
45	3.441000	192.168.1.8	172.217.24.142	TLSv1.2	93	Application Data
46	3.451697	172.217.24.142	192.168.1.8	TCP	54	443 → 52248 [ACK] Seq=1337 Ack=3345 Win=904 Len=0
47	4.926520	148.113.9.138	192.168.1.8	TCP	54	80 → 51929 [ACK] Seq=1 Ack=1 Win=501 Len=0
48	4.926725	192.168.1.8	148.113.9.138	TCP	54	[TCP ACKed unseen segment] 51929 → 80 [ACK] Seq=1 Ack=2 Win=515 Len=0
49	5.557387	192.168.1.8	142.250.66.10	TCP	55	53002 → 443 [ACK] Seq=1 Ack=1 Win=512 Len=1
50	5.566918	142.250.66.10	192.168.1.8	TCP	66	443 → 53002 [ACK] Seq=1 Ack=2 Win=1035 Len=0 SRE=1 SRE=2
51	5.784341	142.250.182.74	192.168.1.8	TLSv1.2	178	Application Data
52	5.831518	192.168.1.8	142.250.182.74	TCP	54	52886 → 443 [ACK] Seq=1 Ack=125 Win=511 Len=0
53	6.466775	zte_13:12:ee	LiteonTechno_e3:80:45	ARP	42	Who has 192.168.1.8? Tell 192.168.1.1
54	6.466818	LiteonTechno_e3:80:45	zte_13:12:ee	ARP	42	192.168.1.8 is at e4:aa:ae:e3:80:45
55	7.946015	192.168.1.8	148.113.9.138	TCP	55	[TCP Keep-Alive] 51929 → 80 [ACK] Seq=0 Ack=2 Win=515 Len=1
56	7.975500	148.113.9.138	192.168.1.8	TCP	66	[TCP Previous segment not captured] 80 → 51929 [ACK] Seq=2 Ack=1 Win=501 Len=0 SRE=1 SRE=2
57	9.259603	192.168.1.8	74.125.24.188	TCP	55	52127 → 5228 [ACK] Seq=1 Ack=1 Win=510 Len=1

Frame 67: 54 bytes on wire (432 bits), 54 bytes captured (432 bits) on interface \Device\NPF...
 Ethernet II, Src: LiteonTechno_e3:80:45 (e4:aa:ae:e3:80:45), Dst: zte_13:12:ee (dc:71:37:13:12:ee)
 Destination: zte_13:12:ee (dc:71:37:13:12:ee)
 ... = LG bit: Globally unique address (factory default)
 ... = IG bit: Individual address (unicast)

6. Create a Filter to display only DHCP packets and inspect the packets

- Select Local Area Connection in Wireshark
- Search DHCP packets in search bar

Student Observation.

1) What is promiscuous mode?

- It is a network mode where a computer's network card captures all traffic, not just the traffic meant for it.
- Useful for tools like Wireshark to analyze all network packets.

2) Does ARP packets have a transport layer header? Explain.

- No transport layer (TCP/UDP) header.
- Works at the data link layer.

3) ~~DNS~~ Transport layer protocol is used by DNS?

- UDP (port 53) for most queries (faster, no connection).
- TCP (port 53) for large data like zone transfers.

4) What is the port number used by HTTP protocol.

The HTTP port number is 80.

5) What is a broadcast IP address?

- Sends to all devices in the network.

Eg:- 192.168.1.255 in Local Network.

RESULT:-

The experiment was successfully performed using Wireshark.

11/11/21

1/9/25

Exp: 5

HAMMING CODE

AIM:

Write a program to implement error detection and correction using HAMMING code concept. Make a test run to input data stream and verify error correction feature.

1) Sender program. (400 lines)

- Convert input text to binary.
- Applies Hamming code to the binary data by adding redundant parity bits.

Saves the Encoded data to a file called channel

2) Receiver program.

- Reads the data from channel.
- Checks for errors using Hamming code.
- If an error is detected, It shows the position of the error.
- If no errors, It removes redundant bits and converts the binary data back to ASCII

CODE

```
import java.util.Scanner;
```

```
public class HammingCode {
```

```
    static void print (int ar[]) {
```

```
        for (int i = 0; i < ar.length; i++) {
```

```
            System.out.print (ar[i]);
```



```

3
    system.out.println();

```

```

3
    static int[] calculation (int[] ar, int r) {
        for (int i = 0; i < r; i++) {
            int x = (int) Math.pow(2, i);
            for (int j = 1; j < ar.length; j++) {
                if (((j >> i) & 1) == 1) {
                    if (x != j)
                        ar[x] = ar[x] ^ ar[j];
                }
            }
        }
    }

```

```

    System.out.println("r" + x + " = " + ar[x]);
    return ar;
}

```

```

3
    static int[] generateCode (String str, int n, int r) {
        int[] ar = new int[r + n + 1];
        int j = 0;

```

```

        for (int i = 1; i < ar.length; i++) {
            if ((Math.ceil(Math.log(i) / Math.log(2)) -
                Math.floor(Math.log(i) / Math.log(2))) == 0) {
                ar[i] = 0;
            } else {
                ar[i] = (int) (str.charAt(j) - '0');
                j++;
            }
        }

```

```

        return ar;
    }
}

```

```

public static void main (String[] args) {
    Scanner sc = new Scanner (System.in);
    System.out.print ("Enter binary data: ");
    String str = sc.nextLine();
    int n = str.length();
    int r = 1;
    while (Math.pow(2, r) < (n + r + 1)) {
        r++;
    }
    int[] ar = generateCode (str, n, r);
    System.out.println ("Generated Hamming Code:");
    ar = calculation (ar, r);
    print (ar);
}
}

```

OUTPUT

Enter Binary data (e.g. 1011): 1111

Generated Hamming code:

$$r_1 = 1$$

$$r_2 = 1$$

$$r_4 = 1$$

11111111

Result :-

Hamming Code Concept is Executed Successfully

11/9/25

Exp: 8

SLIDING WINDOW

AIM:

Write program to implement flow control at data link layer using SLIDING WINDOW PROTOCOL. Simulate the flow of frames from one node to another.

SENDER HOW TO USE :

- Inputs window size & message.
- Sends window-size frames at a time.
- Writes frames to sender-Buffer.
- Receiver reads frames, to Receiver-Buffer.
- Receiver sends ACK or NACK to Sender.
- Sender reads ACK/NACK and continues or resends frames.

You can Manually Edit the files to simulate errors.

CODE

```
import time.
```

```
import random.
```

```
class Sender:
```

```
def __init__(self, total_frames, window_size):
```

```
    self.total_frames = total_frames
```

```
    self.window_size = window_size
```

```
    self.base = 0
```

```
    self.next_seq = 0
```

```
def send_frames(self):
```

```
    print(f"\n[Sender] Total frames to send: {self.total_frames}")
```

```
while self.base < self.total - frames:
```

```
while self.next_seq < self.base + self.window
```

```
size and self.next_seq < self.total:
```

```
print(f"[Sender] Sending Frame {self.next
```

```
seq, self.total - self.next_seq + 1)
```

```
time.sleep(1)
```

```
def ack_received(self, ack):
```

```
print(f"[Sender] Acknowledgment received
```

```
for Frame {ack}")
```

```
if ack >= self.base:
```

```
self.base = ack + 1
```

```
class Receiver:
```

```
def receiver_frame(self, frame_no, sender):
```

```
if random.choice([True, False]):
```

```
print(f"[Receiver] Received Frame {frame_no}")
```

```
sender.ack_received(frame_no)
```

```
else:
```

```
print(f"[Receiver] Receiver {frame_no} lost! No Ack sent.")
```

```
if __name__ == "__main__":
```

```
total_frames = 5
```

```
window_size = 3
```

```
sender = Sender(total_frames, window_size)
```

```
receiver = Receiver()
```

```
sender.send_frames(receiver)
```


OUTPUT

Enter total number of frames : 5

Enter window size : 3

[Sender] Total Frames to send : 5

[Sender] Sending frame 0

[Sender] Sending frame 1

[Sender] Sending frame 2

[Receiver] Successfully received frames 0 to 2.

[Sender] Acknowledgment received for frame 2

[Sender] Sending Frame 3

[Sender] Sending Frame 4

[Receiver] Frame 4 lost or corrupted

[Sender] Timeout - Resending window from Frame 3

[Sender] Sending Frame 3

[Sender] Sending Frame 4

[Receiver] Successfully received frames 3 to 4

[Sender] Acknowledgment received for Frame 4

Transmission Completed.

0.0.0.228 : Host 1, 1.1.1.0 : 109

0.0.0.228 : Host 2, 1.1.1.0 : 109

0.0.0.228 : Host 3, 1.1.1.0 : 109

0.0.0.228 : Host 4, 1.1.1.0 : 109

Result : Sliding window protocol is Executed

Successfully

3/9/20

15/9/25

Exp: 7

Setup & Configure a LAN using

Switch & Ethernet Cables.

AIM: Setup and Configure a LAN using a Switch and Ethernet Cables in your Lab.

Step 1:- plan & Design an appropriate Network topology taking into account Network requirements & Equipment location.

Step 2:- You can take and Computers, a Switch with 8, 16 or 24 ports which is sufficient for Networks of these sizes & 4 Ethernet Cables.

Step 3: Assign IP Addresses.

- On each PC :

- > go to Network & Internet Settings → Ethernet → Properties.

- > Select IPv4 → Use the following IP address

- > Set IP addresses as :

- ↳ PC 1 : 10.1.1.1, Subnet Mask : 255.0.0.0

- ↳ PC 2 : 10.1.1.2, Subnet Mask : 255.0.0.0

- ↳ PC 3 : 10.1.1.3, Subnet Mask : 255.0.0.0

- ↳ PC 4 : 10.1.1.4, Subnet Mask : 255.0.0.0

Step 4: Configure the Switch

- Connect one PC to the Switch.

- Open a browser → Enter Switch IP - login.

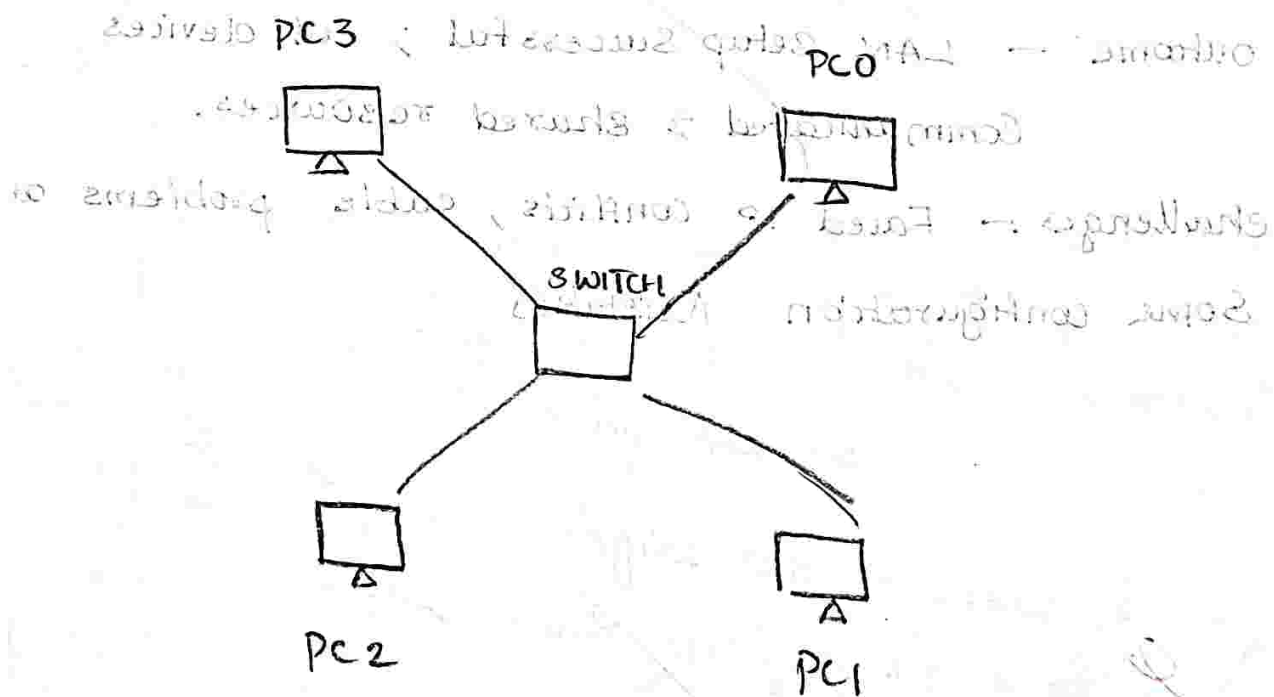
- Assign Switch IP : 10.1.1.5, Subnet Mask : 255.0.0.0

Step 5 : Test Connectivity

- Open Command Prompt on any PC
- Type : ping 10.1.1.5 (to check switch) or ping 10.1.1.2 (to check other PC)

Step 6 :- File Sharing

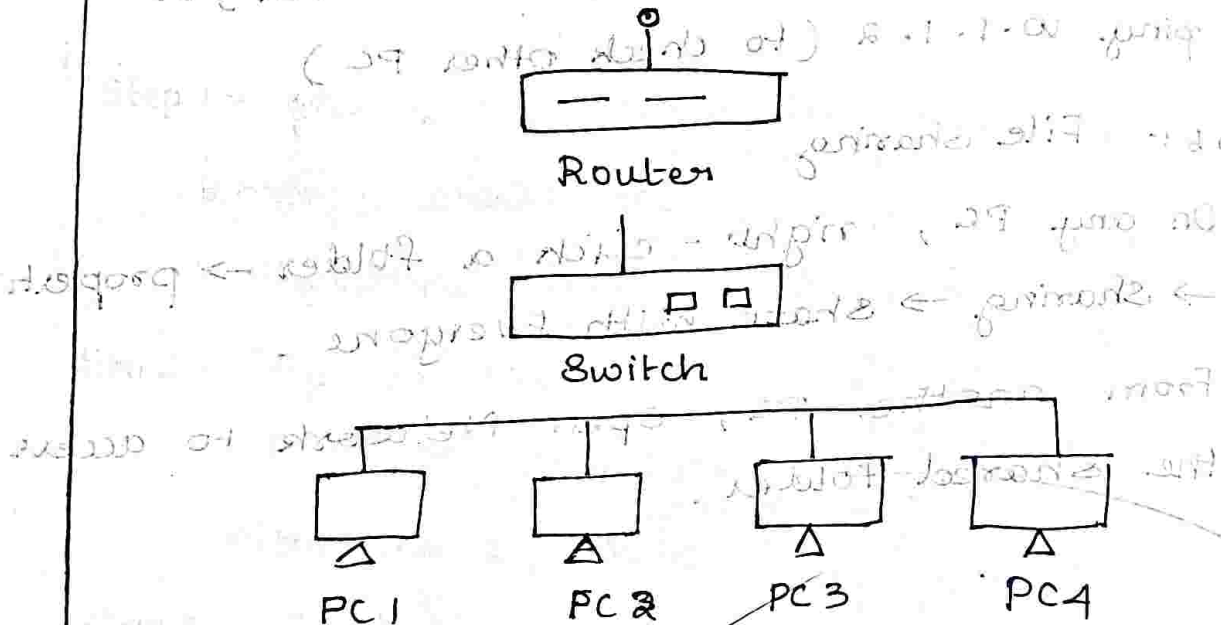
- On any PC, right-click a folder → properties → sharing → share with Everyone -
- From another PC, open Network to access the shared folder.



Step 2: Configure a switch and Ethernet cable for a home network.

Student Observation

- ① Draw neat diagram of the LAN implemented in the Write IP Config of Each device, mention the outcome and challenges faced.



Outcome — LAN Setup Successful ; all devices Communicated & shared resources.

Challenges — Faced IP conflicts, cable problems and Some configuration Mistakes

Result :- Setup & Configure a LAN using a Switch and Ethernet Cable in ~~y~~ is done Successfully.

15/7/25

Exp-8

NMAP on the TryHackme platform.

AIM:-

This Experiment Outline the processes that Nmap takes before port-scanning to find which system are online

- 1) ARP Scan : Uses ARP requests to discover live hosts
- 2) ICMP Scan : This scan uses ICMP requests to identify live hosts
- 3) TCP/UDP Ping Scan : Sends packets to TCP ports and UDP ports to determine live hosts

There will be 2 Scanners introduced

- 1) arp-scan
- 2) mass can

Nmap (Network Mapper) - It is a well-known tool for mapping Networks, locating live hosts and detecting running services.

Nmap's scripting engine can be used to extend its capabilities such as fingerprinting services and Exploiting flaws.

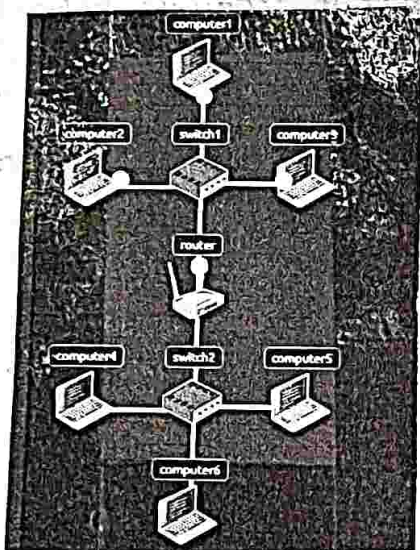
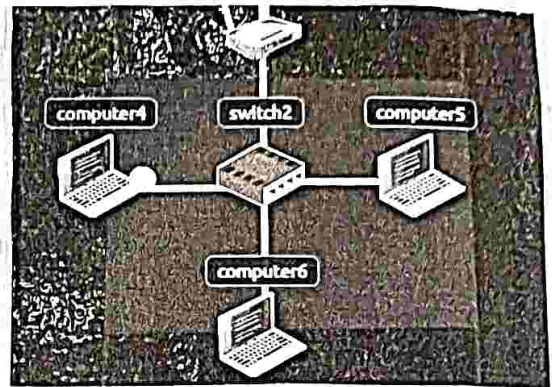
Send Packet

From:

To:

Packet Type:

Data:



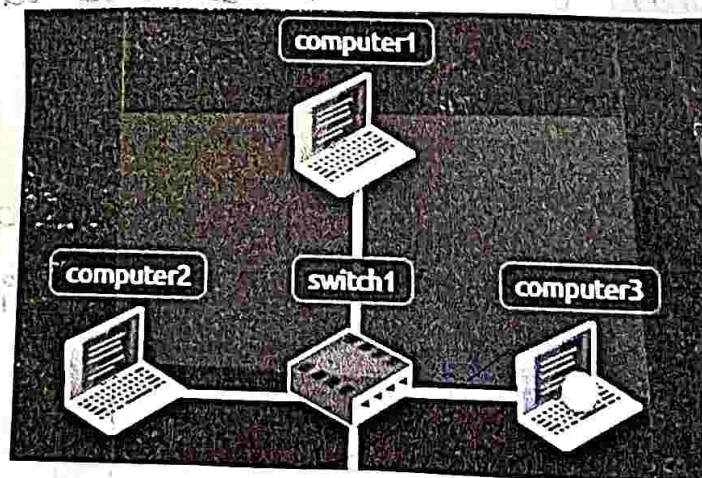
Send Packet

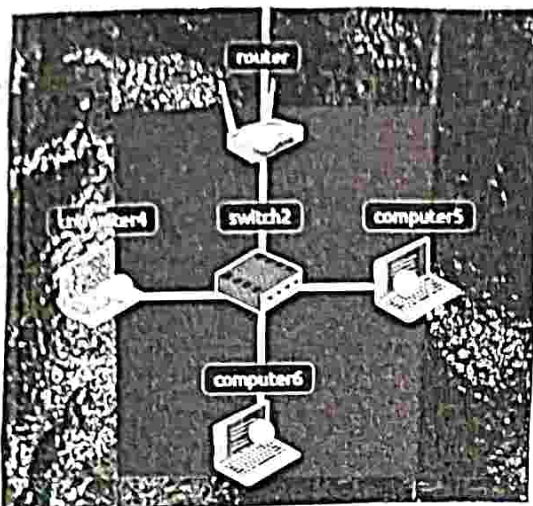
From:

To:

Packet Type:

Data:





```

ARP RESPONSE: Hey computer1, I am
computer2

ARP RESPONSE: Hey computer1, I am
computer3

ARP RESPONSE: Hey computer1, I am
computer2

ARP RESPONSE: Hey computer1, I am
router
  
```

Network Log

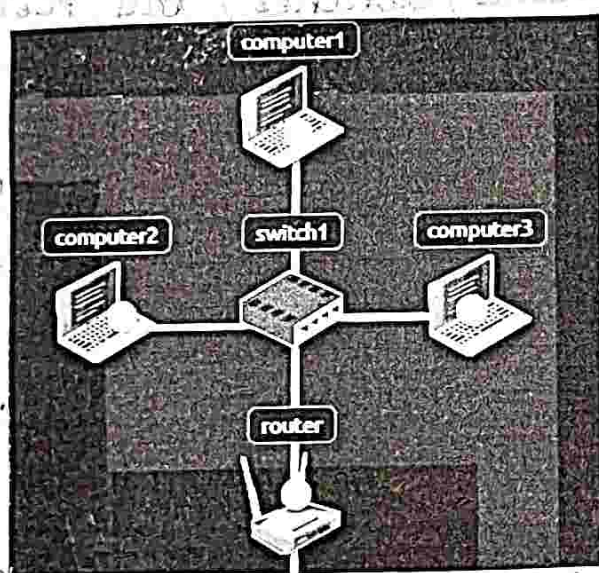
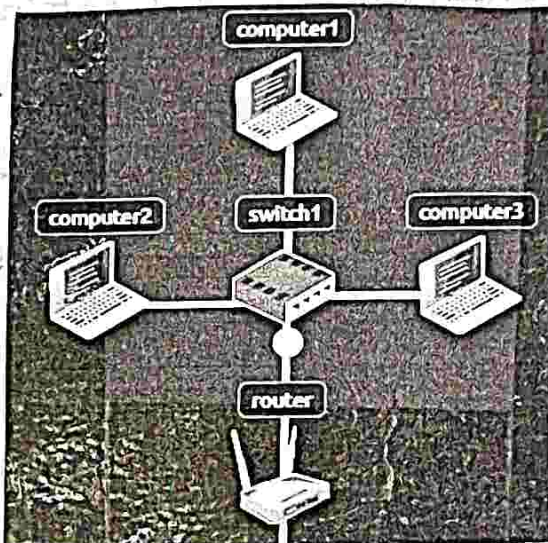
```

PING: Sending Ping Response packet from
computer3 to computer1

PING: computer1 received ping response from
computer3

ARP RESPONSE: Hey computer1, I am
computer2

ARP RESPONSE: Hey computer1, I am
computer3
  
```



Network Log

```

PING: Sending Ping Request packet from
computer1 to computer3

PING: computer3 received ping request from
computer1, sending ping response to
computer1

PING: Sending Ping Response packet from
computer3 to computer1

PING: computer1 received ping response from
computer3
  
```

RESULT :-

The Experiment of NMAP on the Tryhackme platform is done successfully.

29/9/25

Exp - 9

SUBNETTING In CISCO PACKET TRACER.

AIM :

Implementation of Subnetting in Cisco packet Tracer Simulator

CONCEPT :

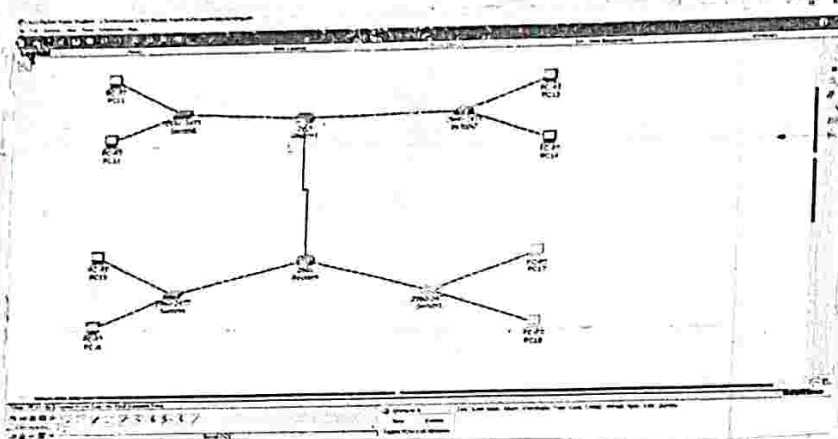
classless IP subnetting allows dividing a network into smaller subnets for efficient IP ~~address utilization~~ ^{usage}. Instead of fixed classful masks, subnetting uses variable-length subnet masks (VLSM) to create networks as per requirements.

STEPS :-

1. Create Topology : Add routers, switches, and PCs in Cisco Packet Tracer.
2. Subnetting : Example - $192.168.1.0 / 24$ Subnetted into $/ 27$ gives 8 subnets ~~IP to routers, switches, and PCs.~~ with 30 hosts each.
3. Assign IPs : Allocate Subnet IPs to routers, switches, and PCs.
Switch 1 & 2 = $FA0/1 : 192.168.1.0 / 27$
PCs :- $192.168.1.11 - 15$
4. Configure Devices : Router = $Gig0/0 : 192.168.1.1$
Router → Configure interfaces with IP & subnet mask.
Switch → set access mode on ports.

PCs → Assign IP, Subnet Mask, and default gateway.

5. Test : Use ping to check connectivity among devices.



STUDENT OBSERVATION :

a) write down your understanding of Subnetting?

Ans:- Subnetting divides a big network into small parts for better control.

b) what is the adv of impl subnetting within a Network?

Ans:- It Improves security, speed, and saves IP address

c) Find out whether subnetting is implemented in your clg?

Ans:- Yes colleges use subnetting. Example →

• Admin : 192.168.1.0/26

• Lab : 192.168.1.64/26

Result:-

Implementation of Subnetting in USCO PACKET Tracer Simulator is done successfully.

[Signature] 2/10/20

3/10/25

Exp 100 INTERNETWORKING WITH ROUTERS IN CISCO PACKET TRACER SIMULATOR.

a) Aim

Internetworking with routers in CISCO PACKET TRACER Simulator, ~~Design~~ Design a simple network with 1 router and 2 PCs using Cisco packet tracer

Procedure:

1. Configure Router:

- > Go to CLI → enable → config t
- > Set IP for FastEthernet 0/0: 192.168.10.1
255.255.255.0 → no shutdown.
- > Set IP for FastEthernet 0/1: 192.168.20.1
255.255.255.0 → no shutdown.

2. Configure PCs:

> PC0: IP 192.168.10.2, Gateway 192.168.10.1

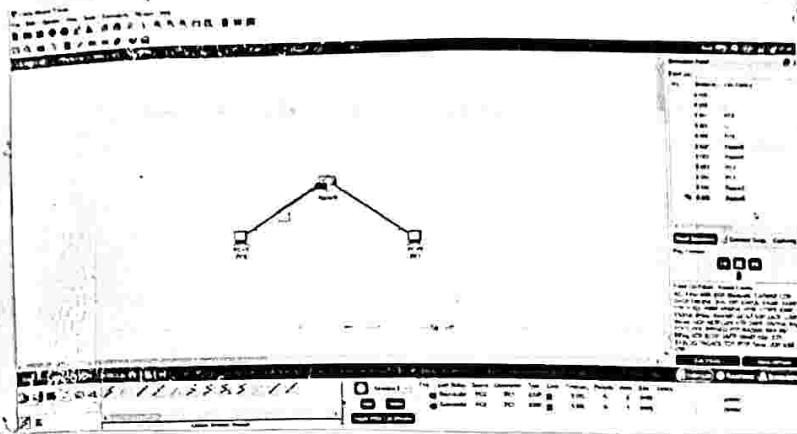
> PC1: IP 192.168.20.2, Gateway 192.168.20.1

3. Connect Devices:

- > PC0 → Router FastEthernet 0/0 (Copper straight-through)
- > PC1 → Router FastEthernet 0/1 (Copper straight-through)

4. Test Connectivity:

- > ping from PC0 to PC1 to verify network connection



b) AIM:

Configure a Network with a wireless router, DHCP server and Internet Connection.

Procedure:

1. Build Network:

- > Add Wireless Router, PC, Laptop, Cable Modem, Internet Cloud, Cisco.com Server.
- > Connect using copper and coaxial cables as required.

2. Configure Wireless Router.

- > Set SSID: HomeNetwork.
- > Enable DHCP & set DNS: 208.67.220.220.

3. Configure Laptop & PC:

- > Laptop: Install wireless module → Connect to Home Network.

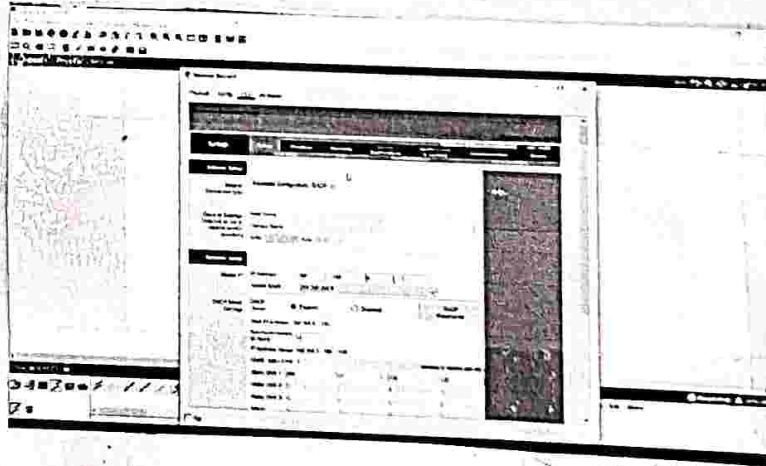
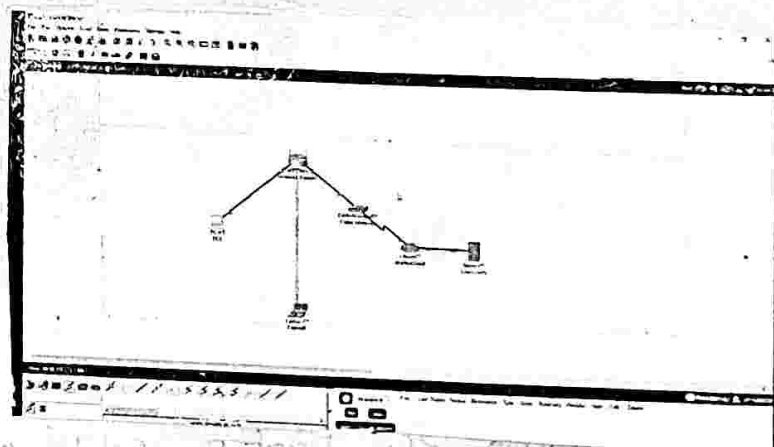
> PC: Enable DHCP → Get IP automatically from router.

4. Configure Cisco . com Server :

- > Enable DHCP & DNS services with IP : 208.67.220.22
- > Set pool & global gateway settings .

5. Verify Connectivity :

- > Refresh IP on PC (ipconfig /release → ipconfig /renew)
- > ping cisco . com to test Internet Connection



STUDENT OBSERVATION

1. Write down the key features of configuring wireless router & DHCP server.

Ans Wireless Router : provides wi-fi , sets SSID , connects device

25/04/2024
DHCP Server: Automatically assigns IP, gateway, subnet,
DNS.

2. what is the significance of DHCP Server in internetworking?

Ans:

- Simplifies IP assignment & avoids conflicts
- Ensures smooth network communication.

3. Design & Configure an inter-network in your lab using switch, router & Ethernet cables. Draw & label.

Ans: > Setup: Router → Switch → PCs (Ethernet cables)

> Eg:-

• Router: 192.168.1.1

• PC0: 192.168.1.2, Gateway: 192.168.1.1

• PC1: 192.168.1.3, Gateway: 192.168.1.1

> Test: ping between PCs to verify connection.

Result:

Inter networking with routers in Cisco packet Tracer simulator is done successfully.

5/10/25

Exp: 11

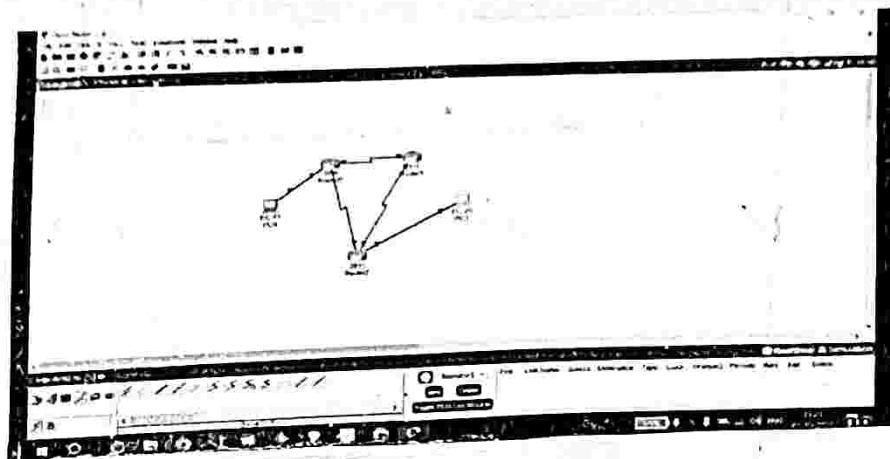
SIMULATE STATIC ROUTING CONFIGURE USING CISCO Packet Tracer.

(a) AIM: Static Routing Configuration

To simulate static routing configuration using Cisco packet Tracer & understand how to Manually add, verify, and Manage static routes in a Network.

PROCEDURE

1. Open Cisco packet Tracer & create 3 routers (R0, R1, R2) with PCs.
2. Connect router and PCs using proper cables.
3. Assign IP address to all interfaces based on the given network table.
4. Configure static routes on each router for Network not directly connected.
5. Verify routing table to check main and backup.
6. Test connectivity using ping & tracer b/w PCs.
7. Simulate link failure by removing a connection and check backup route.
8. Delete static route if needed using router CLI.



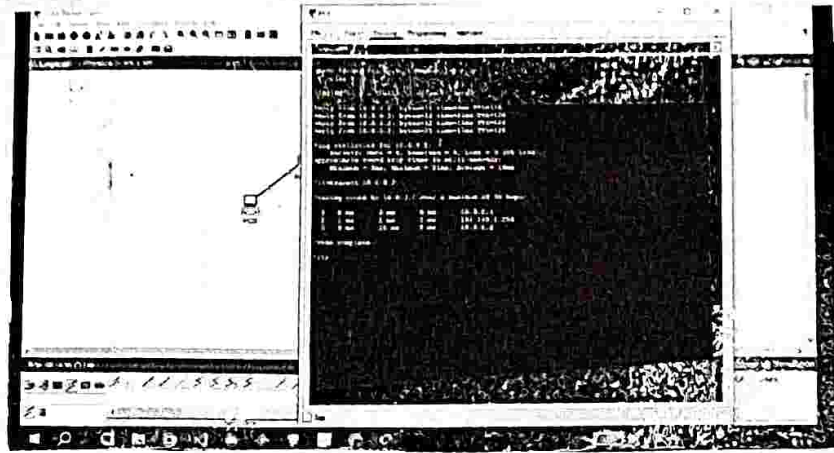
b) AIM : RIP Configuration

To Simulate Routing Information protocol (RIP) using Cisco packet Tracer & understand how routers automatically share routing information within a network.

PROCEDURE

1. Create a Network using 3 routers (R0, R1, R2) & 2 PCs.
2. Connect all devices & assign IP addresses as per table.
3. Enable all interfaces of routers.
4. Enable ^{RIP protocol on} ~~all devices &~~ ~~assign~~ all routers & add network addresses.
5. check routing tables entries learned through RIP
6. ~~Verify network connectivity using ping btw PCs.~~

Correct packet Tracer is done successfully
Date: 10/10/20



AIM : To simulate Static Routing Configuration using Cisco Packet Tracer & understand how routers automatically share routing information within a network.

PROCEDURE

1. Create a network using 3 routers (R0, R1, R2) & 3 PCs.
2. Connect all devices & assign IP addresses as per table.
3. Enable all interfaces of routers.
4. Enable all interfaces of routers & assign IP addresses.
5. Check routing table entries learned through RIP.

Result :

Simulate Static Routing Configuration using Cisco Packet Tracer is done Successfully.

8/10/24

7/10/25

Exp:-12.

Practical 12

AIM:-

- a) Implement Echo client Server using TCP/UDP Sockets

Algorithm :-

```
import time
def ping_server (host = '127.0.0.1', port = 12345):
    with socket.socket(socket.AF_INET, socket.SOCK_DGRAM) as s:
        s.sendto(b'Hello', (host, port))
    except s.timeout:
        print("Request timed Out")
ping_server()
```

Result :-

The Message hello is successfully sent to the Server.

b)

AIM:- Implement chat client server using TCP/UDP Sockets.

Algorithm:-

import socket

def start_server(host = '127.0.0.1', port = 12345):

with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as s:

s.bind((host, port))

s.listen(5)

print(f"UDP Server running on {host}:{port}")

while True:

data, addr = s.recvfrom(1024)

print(f"Received Message from {addr}: {data.decode()}")

start_server()

OUTPUT:

UDP Server running on 127.0.0.1 : 12345

Received Message from ('127.0.0.1', 52345): Hel

The message 'Hello' is successfully sent to the server.

6/10/25

Exp: 13

practical 13

(1000) network . 2 = 1000 1 1000

AIM:

Implement your own ping program.

ALGORITHM:

1. Create a UDP socket.
2. Set a timeout of 2 seconds
3. Record start time
4. Send the message "ping" to the server.
5. Wait to receive a response from the server
 > if received, record end time and display the reply with round-trip time.
 > if timeout occurs, print "Request time out".
6. close the socket.

INPUT:

import socket

import time

def ping_server (host = '127.0.0.1', port = 12345):

with

socket.socket (socket.AF_INET, socket.SOCK_DGRAM)

try:

 s.settimeout(2)

 start = time.time()

s.sendto(b 'Ping', (host, port))

data, addr = s.recvfrom(1024)

end = time.time()

Print(f "Received {data.decode()} from
{addr} in {end - start:.2f} seconds")

except socket.timeout:

print("Request timed out")

if __name__ == "__main__":

ping_server()

OUTPUT:

Received Ping from ('127.0.0.1', 12345) in 0.00 seconds

Request timed out

Result:

Successfully Implement chat Client Server using
TCP/UDP sockets

Algorithm :

- b)
1. Create a UDP socket
 2. Bind the socket to server IP and port
 3. Continuously wait for incoming Messages.
 4. When a message is received :
 - > Display the Message and client address.
 - > Send back "pong" to the client.
 5. Repeat step 3 indefinitely.

INPUT:

```
import socket
```

```
def start_server (host = '127.0.0.1', port = 12345):
```

```
    with
```

```
        socket.socket (socket.AF_INET, socket.SOCK_DGRAM)
```

```
    as s:
```

```
        s.bind ((host, port))
```

```
        print (f"UDP Server running on {host}:{port}")
```

```
    while True:
```

```
        data, addr = s.recvfrom (1024)
```

```
        print (f"Received message from {addr}:  
                {data.decode()}")
```

```
        s.sendto (b'Pong', addr)
```

Algorithm:
if __name__ == "__main__":

start_server()

o/p:-

UDP Server running on 127.0.0.1:12345

Received message from ('127.0.0.1', 52348): ping

> Send back "pong" to the client

> Repeat step 3 indefinitely

INPUT:

import socket

def start_server(host='127.0.0.1', port=12345):

socket = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)

socket.bind((host, port))

while True:

data, addr = socket.recvfrom(1024)

data = data.decode('utf-8')

Result:

Ping program is implemented Successfully

9/10/25
Exp: 14

practical 14

AIM :

write a code using RAW sockets to implement packet sniffing.

ALGORITHM :

1. Open a raw socket in promiscuous mode.
2. Receive a packet (raw bytes) from the socket.
3. Parse Ethernet header to get EtherType.
4. If EtherType == IPV4, parse IP header to get protocol, src IP, dst IP.
5. Optionally parse TCP/UDP/ICMP headers based on IP protocol.
6. Display / log the packet info. Repeat to keep sniffing.

INPUT :

```
from scapy.all import sniff
from scapy.layers.inet import IP, TCP, UDP, ICMP
```

```
def packet_callback(packet):
```

```
    if IP in packet:
```

```
        ip_layer = packet[IP]
```

```
        protocol = ip_layer.proto
```

```
        src_ip = ip_layer.src
```

```
        dst_ip = ip_layer.dst
```


determine the protocol.

protocol_name = ""

if protocol == 1:

protocol_name = "ICMP"

elif protocol == 6:

protocol_name = "TCP"

elif protocol == 17:

protocol_name = "UDP"

else:

protocol_name = "unknown protocol"

print packet details.

print(f"protocol: {protocol_name}")

print(f"Source IP: {src_ip}")

print(f"Destination IP: {dst_ip}")

print("_" * 50)

Capture packets on the default net-work.

sniff(iface = 'Wi-Fi', prn=packet_callback,

filter = 'ip', store=0)

ip_header = packet[14]

protocol = ip_header.protocol

src_ip = ip_header.src

dst_ip = ip_header.dst

OUTPUT:

protocol : TCP

Source IP: 192.168.1.5

Destination IP: 172.217.16.142

Protocol : ICMP

Source IP: 192.168.1.2.

Destination IP: ^{8.8.8.8.}~~224.0.0.251~~

protocol : UDP

Source IP: 192.168.1.10.

Destination IP: ~~172~~ 224.0.0.251

protocol : TCP

Source IP : 192.168.1.5 .

Destination IP : 172.217.16.142

Result:

Successfully executed code using RAW sockets to implement packet sniffing.